Point Decomposition by Root Finding: Replacing Gröbner-Basis Solvers in Elliptic-Curve Index Calculus

with Measurements on Prime and Binary Curves up to NIST P-521

Anonymous

Abstract

The decomposition oracle at the heart of elliptic-curve index calculus asks, for a target point R and a factor base $\mathcal{F}$, whether $R = P_1 + \cdots + P_m$ with every $P_i \in \mathcal{F}$. The textbook answer reduces the question to the roots of Semaev's summation polynomial and hands the resulting polynomial system to a Gröbner-basis engine (F4/F5). We observe that for the summation polynomial S_3 the roots are already known in closed form—they are the x-coordinates of $R \mp P_i$—so the polynomial solve can be replaced by direct evaluation of the group law followed by a table lookup, and for binary curves by a meet-in-the-middle table of pair sums. Both replacements are exact and complete for the decomposition sizes they address, and both admit a fail-closed verification in which every reported decomposition is re-added in the group. We implement them, make root finding the default decomposition strategy in an existing index-calculus codebase, and measure against the same codebase's Gröbner solvers on identical targets: 383–1535× per decomposition on Koblitz curves over $\mathbb{F}_{2^9}$ and $\mathbb{F}_{2^{13}}$ (versus matrix-F4), 3×10^4–5×10^4× on small prime curves, and 3021–5802× on the five NIST prime curves P-192, P-224, P-256, P-384 and P-521 with their standard parameters (versus Buchberger on the same decomposition ideal, in the same field). A dedicated P-256 field implementation gives a further 11–12× over generic arbitrary-precision arithmetic. End to end—complete discrete-log computations with the oracle swapped and everything else fixed, the baseline at its best configuration and msolve as an external F4—the picture is two orders of magnitude more modest and not one-sided: 50–105× on prime curves, 122× on the one binary instance large enough to amortize the pair table, and F4 faster on the three smaller binary instances, where a single solve needs too few decompositions to pay for the table.

We are explicit about what this does and does not change. The result is an improvement in the *constant* of a subroutine—in the evidence vocabulary of the companion repository, classed as *engineering*; the end-to-end speed-ups are real but bounded by the stages the oracle does not touch. It leaves the asymptotic cost of index calculus unchanged, does not alter the exponential size of the Frobenius-invariant factor base on Koblitz curves, does not make prime-field index calculus competitive with Pollard rho, and makes no claim about the security of any deployed curve. All instances are public synthetic or standard-parameter known-answer instances; all code and raw measurements are released.

1 Introduction

Index calculus on elliptic curves, in the form proposed by Semaev [1] and developed by Gaudry [2] and Diem [3], has two expensive stages. The first collects *relations*: random multiples $R = [r]P + [s]Q$ are tested for membership in the m-fold sumset of a factor base $\mathcal{F}$, and each hit is a linear relation among the unknown discrete logarithms of $\mathcal{F}$. The second solves the resulting sparse linear system. The first stage is dominated by the *decomposition oracle*: deciding whether a given R is a sum of m factor-base points and, if so, finding the summands.

The standard route to this oracle is algebraic. Semaev's summation polynomial $S_{m+1}(x_1, \ldots, x_m, x_R)$ vanishes exactly when there exist signs such that $\pm P_1 \pm \cdots \pm P_m = R$ for points with the given x-coordinates. One restricts the x_i to the factor base—either by adjoining vanishing polynomials $\prod_{F \in \mathcal{F}}(x_i - x_F)$ over a prime field, or by Weil descent onto a subspace over an extension field—and solves the resulting zero-dimensional polynomial system. The solver of choice is a Gröbner-basis algorithm: Faugère's F4 [4] or F5 [5], often in a matrix formulation tuned to the system's shape. Substantial

effort has gone into making this solve tractable, through symmetrization, the exploitation of Frobenius and sign symmetries, and careful degree analysis [6, 7, 8].

This paper starts from a simple observation that is well known but, in our experience, under-exploited in implementations: for $m = 2$ the polynomial system does not need to be solved, because its variety is already parametrized. The roots of $S_3(x_R, x_i, X)$ in X are exactly the x-coordinates of $R + F_i$ and $R - F_i$, where F_i is any factor-base point with x-coordinate x_i. One field inversion (amortized across the factor base by Montgomery's trick) and a hash lookup per factor-base element therefore decide the 2-decomposition of R exactly. Over binary fields the same idea takes the form of a meet-in-the-middle table of pair sums, which decides 2-, 3- and 4-decompositions in 1, $|\mathcal{F}|$ and $|\mathcal{F}|^2$ lookups respectively. Neither method approximates; both are complete for the sizes they address, and both report a witness that can be re-added in the group.

Contributions.

1. We state the two root-finding decomposition procedures precisely (§3), prove their exactness (Proposition 1), and give their cost in field operations and lookups.
2. We implement both in an existing Rust index-calculus codebase that already contained matrix-F4 and Buchberger decomposition solvers, make root finding the default strategy, and keep the Gröbner solvers as selectable baselines (§4).
3. We define a fail-closed verification protocol under which every reported decomposition is re-added in the group, cross-checked against the summation polynomial, and—where a Gröbner basis is also computed—checked to lie in the variety of that basis (§5).
4. We measure root finding against the Gröbner solvers on identical targets (§6): on binary Koblitz curves, on small prime curves, on prime fields from 40 to 83 bits, and on the five NIST prime curves P-192 through P-521 with their standard parameters, including a per-curve specialized field for P-256.
5. We state precisely what the measurements do and do not imply (§7). In particular, the asymptotics of index calculus are unchanged, the exponential factor-base wall on Koblitz curves is untouched, and no security claim about any deployed curve follows.

2 Background

2.1 Summation polynomials and the decomposition problem

Let $E/\mathbb{F}_q$ be an elliptic curve. Semaev [1] defined the summation polynomials $S_m \in \mathbb{F}_q[x_1, \dots, x_m]$ by the property that for $x_1, \dots, x_m \in \overline{\mathbb{F}}_q$,

$$S_m(x_1, \dots, x_m) = 0 \iff \exists\, P_1, \dots, P_m \in E(\overline{\mathbb{F}}_q) :\ x(P_i) = x_i,\ \ P_1 + \cdots + P_m = \mathcal{O}.$$

For short Weierstrass $y^2 = x^3 + ax + b$ the third polynomial is

$$S_3(x_1, x_2, x_3) = (x_1 - x_2)^2 x_3^2 - 2\big[(x_1 + x_2)(x_1 x_2 + a) + 2b\big]x_3 + \big[(x_1 x_2 - a)^2 - 4b(x_1 + x_2)\big], \qquad (1)$$

and higher S_m are obtained by resultants, with $\deg_{x_i} S_m = 2^{m-2}$.

Given a factor base $\mathcal{F} \subset E(\mathbb{F}_q)$ and a target R, the *m-decomposition problem* is to decide whether there exist $F_1, \dots, F_m \in \mathcal{F}$ and signs $\epsilon_i \in \{\pm 1\}$ with $R = \sum_i \epsilon_i F_i$, and to exhibit them. Since S_{m+1} only sees x-coordinates, a solution of $S_{m+1}(x_1, \dots, x_m, x(R)) = 0$ with each $x_i \in x(\mathcal{F})$ gives a decomposition up to signs, which are then recovered by evaluating the group law.

2.2 The Gröbner route

Over a prime field $\mathbb{F}_p$ the factor base is typically the set of points with x-coordinate in an interval, and the decomposition system is

$$I_R = \big\langle\, S_{m+1}(x_1, \dots, x_m, x_R),\ V(x_1),\ \dots,\ V(x_m)\,\big\rangle, \qquad V(X) = \prod_{F \in \mathcal{F}} \big(X - x(F)\big). \qquad (2)$$

I_R is zero-dimensional; R is m-decomposable iff $I_R \neq \langle 1 \rangle$, and a Gröbner basis of I_R in lexicographic order exposes the solutions. Over $\mathbb{F}_{2^n}$ one instead restricts the x_i to an $\mathbb{F}_2$-subspace V of dimension ℓ, writes each x_i in a basis of V, and Weil-descends S_{m+1} to a system of n Boolean equations in $m\ell$ Boolean unknowns, again solved by F4/F5 with field equations adjoined [6, 7].

In both settings the Gröbner solve is the step that dominates the per-target cost, and in both settings the solve is doing real work: the variety of I_R is not given to it, it has to be found. The point of this paper is that for small m the variety is given, by the group law.

2.3 What index calculus does and does not achieve

Two facts frame everything that follows. Over prime fields, with $|\mathcal{F}| \approx p^{1/m}$, the probability that a random R decomposes is about $1/m!$, and the number of relations needed is about $|\mathcal{F}|$; the resulting cost is $\Omega(p^{1/m} \cdot m!)$ decompositions plus the cost of each decomposition, and Gaudry [2] and Diem [3] showed this does not beat Pollard rho [13] on prime-field curves for any m once the decomposition cost is accounted for. Over $\mathbb{F}_{2^n}$ with n prime, the Frobenius-invariant factor base relevant to Koblitz curves has dimension $\ell = \mathrm{ord}_2(n)$ and size 2^ℓ, which for the cryptographic n of interest is itself exponential. Faster decomposition changes neither fact. We return to this in §7.

3 Root-Finding Decomposition

3.1 Prime curves: the S_3 roots are $x(R \mp F_i)$

Proposition 1. *Let $E : y^2 = x^3 + ax + b$ over $\mathbb{F}_p$, $p > 3$, let $R = (x_R, y_R)$ and $F = (x_F, y_F)$ be affine points with $x_R \neq x_F$. Then the two roots of $X \mapsto S_3(x_R, x_F, X)$ in $\overline{\mathbb{F}}_p$ are exactly $x(R+F)$ and $x(R-F)$, counted with multiplicity.*

Proof. By the defining property of S_3, $S_3(x_R, x_F, X) = 0$ iff there exist $P, Q, S \in E(\overline{\mathbb{F}}_p)$ with $x(P) = x_R$, $x(Q) = x_F$, $x(S) = X$ and $P + Q + S = \mathcal{O}$. The points with x-coordinate x_R are $\pm R$, those with x-coordinate x_F are $\pm F$, so $S = -(\pm R \pm F)$ and $X \in \{x(R+F), x(R-F)\}$. The polynomial has degree 2 in X since $x_R \neq x_F$ makes the leading coefficient $(x_R - x_F)^2$ non-zero, so these are all the roots; if $x(R+F) = x(R-F)$ the root is double. □

Remark 1 (Machine-checked form). Proposition 1 is formalized in Lean 4 against Mathlib as the factorization identity $S_3(x_R, x_F, X) = (x_R - x_F)^2\,(X - x_+)(X - x_-)$ for every $X \in K$, where $x_\pm = \lambda_\pm^2 - x_R - x_F$ and $\lambda_\pm = (\pm y_F - y_R)/(x_F - x_R)$ are the affine chord formulas for $x(R \pm F)$, over an arbitrary field K with the two curve equations as hypotheses (`SemaevS3.S3_factor`); the root statement $S_3(x_R, x_F, X) = 0 \iff X \in \{x_+, x_-\}$ follows (`SemaevS3.S3_root_iff`). The proof clears the two chord denominators and closes a polynomial identity modulo the curve equations with explicit cofactors; it depends only on Lean's standard axioms. The source and a pinned Mathlib manifest are in `papers/rootfinding-decomposition/lean/` (§9).

Proposition 1 says that the univariate root-finding problem one would solve for each factor-base element in a "root-finding" index calculus is trivial: the roots are produced by one slope computation. Algorithm 1 makes this concrete, with Montgomery's batched-inversion trick [14] so that a sweep over the whole factor base costs a single field inversion.

Correctness and completeness. If $R = \epsilon F_i + \tau F_j$ with $x_i \neq x_R$, then $R - \epsilon F_i = \tau F_j$ has x-coordinate $x_j \in x(\mathcal{F})$, so the lookup at iteration (i, ϵ) succeeds and the recovered y_3 fixes τ. Conversely every returned tuple satisfies $R - \epsilon F_i = \tau F_j$ by construction of (x_3, y_3). The only excluded case $x_i = x_R$ means $R = \pm F_i$, which is a 1-decomposition and is handled by a direct lookup before the sweep.

Cost. One field inversion, $3(N-1)$ multiplications for the batch inverse, and per factor-base element two slope evaluations (each one multiplication, one squaring, a handful of additions) and two hash lookups: $\Theta(N)$ field multiplications and $2N$ lookups, with $N = |\mathcal{F}|$. No polynomial arithmetic, no square roots, no linear algebra. By contrast the Gröbner route computes a basis of the ideal (2), whose vanishing polynomials already have degree N.

Algorithm 1 Prime-curve 2-decomposition by direct subtraction

Require: Curve $E : y^2 = x^3 + ax + b$ over $\mathbb{F}_p$; factor base $\mathcal{F} = \{F_1, \ldots, F_N\}$ with $F_i = (x_i, y_i)$; hash map $\mathsf{idx} : x(\mathcal{F}) \to \{1, \ldots, N\}$; target $R = (x_R, y_R)$.
Ensure: (i, ϵ, j, τ) with $R = \epsilon F_i + \tau F_j$, or $\perp$ if no such pair exists.

$D \leftarrow [\, x_R - x_i : x_R \neq x_i \,]$ ▷ non-zero differences
$D^{-1} \leftarrow \textsc{BatchInverse}(D)$ ▷ one inversion, $3(N-1)$ mults
for each i with $x_R \neq x_i$ **do**
  for $(\epsilon, \nu) \in \{(+1,\, y_R + y_i),\ (-1,\, y_R - y_i)\}$ **do**
    $\lambda \leftarrow \nu \cdot (x_R - x_i)^{-1}$ ▷ slope of the line through R and $-\epsilon F_i$
    $x_3 \leftarrow \lambda^2 - x_R - x_i$ ▷ $= x(R - \epsilon F_i)$
    if $x_3 \in \operatorname{dom} \mathsf{idx}$ **then**
      $j \leftarrow \mathsf{idx}(x_3)$; $y_3 \leftarrow \lambda(x_R - x_3) - y_R$
      $\tau \leftarrow +1$ if $y_3 = y_j$ else -1
      return (i, ϵ, j, τ)
    end if
  end for
end for
return $\perp$

Larger m. The same idea extends to $m = 3$ at cost $\Theta(N^2)$ by sweeping pairs (F_i, F_j) and looking up $x(R - \epsilon F_i - \epsilon' F_j)$, and in general to m at $\Theta(N^{m-1})$. This is a meet-in-the-middle search rather than an algebraic solve, and the trade-off is explicit: its cost is exponential in m with base N, whereas Gröbner methods aim at a cost polynomial in N for the structured systems that arise. In the regimes measured here, $m = 2$, the comparison is not close. We do not claim it stays one-sided for large m; see §7.

3.2 Binary curves: a meet-in-the-middle pair-sum table

Over $\mathbb{F}_{2^n}$ the factor base of interest in the Koblitz setting is a Frobenius-invariant set $\mathcal{F}$ of size N, and the targets are m-decomposed with m up to 4. Rather than Weil-descend S_{m+1}, we precompute the table

$$T = \left\{\, x(F_i + F_j) \mapsto (i, j) \;:\; 1 \le i \le j \le N \,\right\},$$

of size $\binom{N+1}{2}$, once per factor base. Then (Algorithm 2):

- $m = 2$: R decomposes iff $x(R) \in \operatorname{dom} T$. One lookup.
- $m = 3$: R decomposes iff $x(R - F_k) \in \operatorname{dom} T$ for some k. N lookups.
- $m = 4$: R decomposes iff $x(R - F_k - F_l) \in \operatorname{dom} T$ for some $k \le l$. $\binom{N+1}{2}$ lookups.

Signs are handled as in the prime case, by trying both $\pm F_k$ and checking the recovered y-coordinate against the table entry. The table is the "left half" of a birthday search; the right half is enumerated per target. The procedure is exact and complete for each m because it enumerates every way to write R as (table entry) + (at most $m - 2$ further summands).

Cost. Table construction is $\binom{N+1}{2}$ point additions, paid once per factor base and amortized over all targets. Per target the cost is 1, $2N$ or $\sim 2N^2$ group operations and lookups for $m = 2, 3, 4$. The matrix-F4 route pays a Macaulay-matrix reduction per target whose size depends on the Weil-descended system, not on N directly; the measured comparison is in §6.1.

3.3 Relationship to the Gröbner basis

It is worth being precise about what is and is not being replaced. The ideal I_R of (2) is zero-dimensional and its variety $V(I_R)$ is the set of x-coordinate tuples of m-decompositions of R. A Gröbner basis of I_R is a canonical generating set from which $V(I_R)$ can be read off. Algorithms 1 and 2 compute $V(I_R)$ *directly*, by enumeration guided by the group law, without computing a generating set of I_R at all. Both approaches answer the same question and, on the same instance, must agree; §5 turns this into a check. The Gröbner basis carries more information—it certifies non-decomposability as $I_R = \langle 1 \rangle$, and the degree data it exposes underlie the complexity analyses of [6, 7]—which is why we keep it available as a baseline rather than deleting it.

Algorithm 2 Binary-curve m-decomposition by pair-sum table ($m \in \{2, 3, 4\}$)

Require: Factor base $\mathcal{F} = \{F_1, \ldots, F_N\}$ over $\mathbb{F}_{2^n}$; pair table $T : x(F_i + F_j) \mapsto (i, j)$ for $i \le j$; target R; m.
Ensure: Indices $(i_1, \ldots, i_m)$ and signs with $R = \sum \pm F_{i_k}$, or $\perp$.

$\mathcal{S} \leftarrow \{\mathcal{O}\}$ if $m = 2$; $\{\pm F_k\}$ if $m = 3$; $\{\pm F_k \pm F_l : k \le l\}$ if $m = 4$ ▷ right half, enumerated lazily
for each $S \in \mathcal{S}$ **do**
  $R' \leftarrow R - S$
  if $x(R') \in \operatorname{dom} T$ **then**
    $(i, j) \leftarrow T(x(R'))$
    fix signs by comparing $y(R')$ with $y(F_i + F_j)$
    return (i, j) together with the indices of S
  end if
end for
return $\perp$

4 Implementation

The algorithms were implemented in an existing Rust codebase for elliptic-curve index calculus that already provided (i) a Koblitz-curve index-calculus pipeline with a matrix-F4 decomposition solver over the Weil-descended Semaev system and (ii) a generic Buchberger implementation over multivariate polynomials with arbitrary-precision coefficients. We added:

- `prime_fast`: a prime-curve engine with a 64-bit Montgomery field ($p < 2^{62}$), affine points, factor-base construction by lifting consecutive x-coordinates, batched inversion, and the direct-subtraction decomposition of Algorithm 1.
- `PairSumTable`: the table of Algorithm 2 and its $m \in \{2, 3, 4\}$ decomposition, over the codebase's packed $\mathbb{F}_{2^n}$ representation ($n \le 63$).
- A `DecompositionStrategy` enumeration in the Koblitz pipeline, with `PairTable` now the default and `Groebner` retained as a selectable baseline.
- An arbitrary-precision (`BigUint`) instance of Algorithm 1 for fields past the native word size, and a P-256-specific instance over the codebase's constant-time Montgomery field for that prime.

The last point is the "specialization per curve" that the NIST measurements call for. Each NIST prime has a Solinas form admitting a dedicated fast reduction; the codebase already contained such a field for P-256, and the oracle is generic over the field, so instantiating it there was a matter of supplying the field operations. We did not write new Solinas fields for P-192, P-224, P-384 or P-521; those curves are measured on the generic field, and their specialization is the same mechanical step. Nothing in the algorithm depends on the curve beyond $a = -3$, which all five share.

5 Fail-Closed Verification

Every measurement in this paper runs under the following protocol. A decomposition that fails any check aborts the run; no row in any table below was produced by a run with a failing check.

1. **Group re-addition.** Every reported tuple (i, ϵ, j, τ) is re-evaluated as $\epsilon F_i + \tau F_j$ with an independent affine addition routine and compared to R in both coordinates.
2. **Summation-polynomial check.** For every reported pair, $S_3(x_R, x_i, x_j)$ is evaluated with the codebase's existing S_3 routine and required to be zero.
3. **Variety membership.** Where a Gröbner basis of I_R is also computed (the head-to-head experiments), every element of the reduced basis is evaluated at (x_i, x_j) and required to vanish, and the basis is required not to be $\{1\}$.
4. **Agreement on refutation.** Where the root finder reports $\perp$ and a Gröbner basis is computed, the reduced basis is required to be $\{1\}$. Where the root finder reports $\perp$ on a target constructed as $F_i \pm F_j$, the run aborts.

5. **Positive controls.** Each experiment includes targets constructed as $F_i \pm F_j$ for random $i \neq j$ that must decompose, and random on-curve targets whose verdicts are recorded.

Checks 3 and 4 are what make the Gröbner baseline more than a timer: they establish on every target that the two methods compute the same variety.

6 Experiments

All timings are single-threaded wall-clock on a desktop machine, in release builds, with the Gröbner baselines taken from the same codebase as the root finders so that field arithmetic, data structures and allocator are shared. Every number below is read from a committed artefact in the companion repository, named in the table caption with the repository prefix `crypto:` (§9). Per-target ratios in §6.1–§6.4 are *stage* measurements of the decomposition oracle; §6.5 measures complete discrete-log computations, every one of which recovered and verified the planted logarithm. Ratios are stable to within about 2× across runs; absolute times vary more on a shared machine, and we note the one place this is visible.

6.1 Binary Koblitz curves: pair table versus matrix-F4

Koblitz curves $K_0 : y^2 + xy = x^3 + 1$ over $\mathbb{F}_{2^n}$ with Frobenius-invariant factor bases; $m = 2$; all targets constructed as $F_i + F_j$ so that every one is decomposable and both oracles must find it. Table 1 reports per-decomposition time.

Table 1: Binary Koblitz $m = 2$ decomposition, per target. Both oracles agreed on every target. Artefact: `crypto:research/semaev_root_vs_groebner_20261012/results.json` (`kind=binary`; totals divided by target count).

curve	$\lvert\mathcal{F}\rvert$	targets	pair table	matrix-F4	factor
$K_0/\mathbb{F}_{2^9}$	55	254	2.9 μs	1.10 ms	383×
$K_0/\mathbb{F}_{2^{13}}$	4005	256	9.9 μs	15.3 ms	1535×

Per-target is not end-to-end. The per-decomposition numbers in Table 1 isolate the oracle cost, which is the quantity the Gröbner solve dominated. They do not by themselves say what a complete discrete-log computation pays, because the pair table is built once and a small instance may not need enough decompositions to amortize it. §6.5 measures that directly, and on the smallest binary instances F4 wins.

6.2 Small prime curves: direct subtraction versus Buchberger

Curves $y^2 = x^3 - 3x + b$ over 16-, 20- and 24-bit primes with prime group order; $|\mathcal{F}| = 6$ (the Buchberger baseline is impractical much past this); 40 targets $[k]G$, $k = 1, \ldots, 40$, per size. The Gröbner baseline computes a reduced Gröbner basis of I_R in graded reverse-lexicographic order. Table 2 reports per-target time. In each size exactly one of the 40 targets was decomposable and the remaining 39 produced the unit ideal from Buchberger, matching the root finder's $\perp$.

Table 2: Prime $m = 2$ decomposition, $|\mathcal{F}| = 6$, per target. Artefact: `crypto:research/semaev_root_vs_groebner_20261012/results.json` (`kind=prime`; totals divided by 40).

bits	targets	decomposed / refuted	direct subtraction	Buchberger	factor
16	40	1 / 39	0.74 μs	26.3 ms	35592×
20	40	1 / 39	0.83 μs	28.7 ms	34602×
24	40	1 / 39	0.65 μs	30.6 ms	46911×

The root finder was also run at factor-base sizes where the Gröbner baseline is infeasible, as the decomposition oracle inside a full index-calculus discrete-log computation, recovering the correct logarithm at 20 and 24 bits with $|\mathcal{F}| = 120$ (4 438 and 78 159 targets in 9 ms and 163 ms) and at 28 bits with $|\mathcal{F}| = 160$ (1 040 671 targets in 2.83 s; artefact: same file, `kind=prime_scaling`, `log_ok=true`).

6.3 Scaling the prime root finder

To separate the method from the word size, Algorithm 1 was run on the native 64-bit Montgomery field at 40, 48, 56 and 61 bits with $|\mathcal{F}| = 120$, and on the arbitrary-precision field at $p = 2^{83} - 55$ with $|\mathcal{F}| = 80$. Table 3 reports per-target time on 200 random on-curve targets and the verification count on constructed positive controls.

Table 3: Prime root finder across field sizes. "verified" counts constructed $F_i + F_j$ controls that decomposed and re-added correctly. Artefact: `crypto:research/semaev_rootfind_scaling_20261012/results.json`.

field	bits	$\lvert\mathcal{F}\rvert$	per target	verified
native $u64$ Montgomery	40	120	2.95 μs	200/200
	48	120	3.87 μs	200/200
	56	120	3.35 μs	200/200
	61	120	8.82 μs	200/200
arbitrary precision	83	80	5.99 ms	100/100

The native path is flat to the 2^{62} ceiling. The 83-bit row is three orders of magnitude slower because that first arbitrary-precision instance performed one un-batched modular inversion per factor-base element; the NIST instances below use the batched form and recover most of the gap. The point of the row is that the algorithm, not the word size, is what extends.

On the binary side the pairs-and-solve root finder over a bounded subspace of $\mathbb{F}_{2^{53}}$ was cross-checked against exhaustive S_4 search on 200/200 targets at subspace dimension $\ell = 6$, and its rejection cost measured at $\ell = 8, 10, 11$ (38 ms, 585 ms, 2.48 s per target, growing with $|V| = 2^\ell$ as expected; artefact: `crypto:research/semaev_rootfind_scaling_20261012/console.txt`). The subspace is bounded on purpose; see §7.

6.4 NIST prime curves P-192 through P-521

The five NIST prime curves [15] were instantiated with their standard p and b ($a = -3$ throughout); the factor base is the first N points obtained by lifting $x = 1, 2, \ldots$. Three measurements were made per curve.

A. Head-to-head against Buchberger. $|\mathcal{F}| = 6$; 10 targets per curve, 5 constructed as $F_i + F_j$ and 5 random on-curve points. Both methods run in the same arbitrary-precision field. All checks of §5 passed on every target: all 5 constructed targets decomposed and verified on every curve, all 5 random targets were refuted by both methods on every curve, and the root finder's witnesses lay in the variety of the reduced Gröbner basis in every case.

Table 4: NIST P-curves, $m = 2$, $|\mathcal{F}| = 6$, 10 targets each, same field and ideal. All fail-closed checks passed. Artefact: `crypto:research/semaev_rootfind_nist_20261012/results.json` (`section=vs_f4`).

curve	bits	direct subtraction	Buchberger	factor	decomposed / refuted
P-192	192	120 μs	414 ms	3455×	5 / 5
P-224	224	218 μs	1.27 s	5802×	5 / 5
P-256	256	203 μs	613 ms	3021×	5 / 5
P-384	384	337 μs	1.09 s	3235×	5 / 5
P-521	521	747 μs	2.54 s	3395×	5 / 5

The factor is roughly flat in the field size. Both methods pay the same arbitrary-precision arithmetic per operation; what separates them is that the root finder performs $\Theta(|\mathcal{F}|)$ field operations while the Gröbner solve performs a basis computation whose coefficient arithmetic is all in that field.

B. Root-finder scaling at $|\mathcal{F}| = 64$. 200 random on-curve targets (the rejection path, which dominates in practice) and 100 constructed controls $F_i \pm F_j$, every control re-added in the group and checked

against S_3.

Table 5: NIST P-curves, root finder only, $|\mathcal{F}| = 64$, generic arbitrary-precision field. Artefact: same file, `section=scale_generic`.

curve	rejection / target	controls verified	control / target
P-192	369 μs	100/100	301 μs
P-224	428 μs	100/100	268 μs
P-256	433 μs	100/100	324 μs
P-384	583 μs	100/100	394 μs
P-521	1.28 ms	100/100	3.66 ms*

*Run-to-run jitter on a shared machine; the preceding run measured 1.13 ms. The ratios in Table 4 are stable to within $\sim 2\times$.

C. Per-curve specialization on P-256. The identical oracle, with the identical batched inversion, instantiated over the codebase's dedicated constant-time Montgomery field for P-256.

Table 6: P-256, $|\mathcal{F}| = 64$: generic field versus dedicated field. Both paths agreed on every target. Artefact: same file, `section=scale_specialised`.

	generic	P-256 field	factor
rejection / target	433 μs	41 μs	11$\times$
control / target	324 μs	28 μs	12$\times$

Closing check. The benchmark asserts that the root finder's total time is below Buchberger's on every one of the five curves with all verification checks passing, and exits non-zero otherwise. It passed.

6.5 End to end: complete discrete-log computations

The question a user actually asks is not how fast the oracle is per target but how long a complete discrete-log computation takes with one oracle versus the other. This section measures that, with the Gröbner baseline given every advantage we could give it, and reports every row—including the ones the baseline wins.

Protocol. Each row is a complete index-calculus solve of a planted logarithm d: factor base, (pair table), relation collection on a seeded target stream, linear algebra modulo the group order, and the group check $[d]G = Q$. The oracle is swapped and everything else is held fixed; the two oracles see the identical target stream. *All times are CPU time* (user+system of the process and its reaped children, via `getrusage`), not wall-clock: the benchmark is single-threaded, the external solver runs as a child process, and CPU time is immune to the scheduling delay of a shared machine, which would otherwise inflate long solves far more than microsecond lookups and flatter the root finder.

The baseline's advantages.

- *Binary.* Matrix-F4 at the best `max_degree` found by a per-n sweep (2 was best at every n); root finding is charged its one-time pair-table build inside every single-solve number.
- *Prime.* Two Gröbner baselines on the identical ideal (2): the codebase's Buchberger at whichever of grevlex/lex decides faster on a calibration sample (lex, at every size), and **msolve** [11]—a state-of-the-art F4 implementation—as an external process. msolve is charged only its own process CPU: each decide is paired with an adjacent run on a trivial ideal, alternating order, and the mean paired difference is what is charged, so process start-up and file I/O are excluded. A per-decide value is reported only when it clears two standard errors of the paired mean; otherwise the 2σ bound is charged (in the baseline's favour). On a hit, the witness is read off by direct subtraction and that time is added to msolve's account; this undercharges F4, since reading a solution off a basis it has already computed is cheap next to computing it.

- *Budget rule.* A baseline is measured in full when its projected time is under the run's budget; otherwise it is *composed*—a sampled per-target cost on 100 (Buchberger) or 400 (msolve) targets from the same stream, times the root finder's measured target count—and is labelled COMPOSED wherever it appears. Root finding is always measured in full.

Binary Koblitz curves. Three seeds per strategy, same Q; Table 7.

Table 7: Complete Koblitz DLP, $m = 2$, CPU time, mean of three seeds. "single" charges root finding for its table build; "extra" is the ratio for each additional solve against the same factor base (table already built). Every solve verified. Artefact: `crypto:research/semaev_rootfind_e2e_20261012/run1_results.json` (`section=binary_e2e`).

n	$\lvert\mathcal{F}\rvert$	$\log_2 r$	root finding	table build	matrix-F4	single	extra
5	21	4	4.3 ms	1.4 ms	0.3 ms	F4 faster (0.06×)	F4 faster
9	55	7	3.9 ms	1.5 ms	1.7 ms	F4 faster (0.45×)	F4 faster (0.7×)
13	4005	11	298 ms	260 ms	14.2 ms	F4 faster (0.05×)	F4 faster (0.4×)
23	2025	21	90.8 ms	89.2 ms	11.1 s	**122**×	$\gtrsim 10^3\times$[1]

Why F4 wins three of four. At $n \le 13$ the subgroup is tiny ($r < 2^{11}$), so a solve needs only a few hundred decompositions, and matrix-F4 at `max_degree` 2 on these small Weil-descended systems costs tens of microseconds per target: the whole F4 solve is cheaper than building $\binom{\lvert\mathcal{F}\rvert+1}{2}$ pair sums that are then barely used. At $n = 23$ the subgroup is 2^{21}, the descended system is larger, F4 costs seconds per solve, and the table—built in 89 ms—pays for itself in the first solve. The crossover is in the size of the instance, not in the oracle: for a factor base used across many solves the table is amortized and root finding wins at every n measured by the per-decomposition numbers of Table 1, but a single tiny solve is not that regime.

Prime curves. Table 8. The 16-bit row is measured in full against both baselines. The composed rows are labelled.

Table 8: Complete prime-curve DLP, $m = 2$, CPU time; all three oracles on the identical seeded target stream. Every solve verified. Artefact: `crypto:research/semaev_rootfind_e2e_20261012/run2/results.json` (`section=prime_e2e`).

bits	$\lvert\mathcal{F}\rvert$	targets	root finding	Buchberger	msolve (F4)	vs msolve
16	8	6 544	23.1 ms	133 s (5767×)	2.31 s	**100**×
20	12	75 683	416.1 ms	6 614 s COMP. (15895×)	43.77 s COMP.	**105**×
24	12	1 501 376	12.30 s	134 492 s COMP. (10937×)	619 s COMP.	**50**×

Each method at its own best factor-base size. Fixing $\lvert\mathcal{F}\rvert$ across methods is itself a choice that can favour either side: F4's per-decide cost grows with $\lvert\mathcal{F}\rvert$ (the vanishing polynomials have degree $\lvert\mathcal{F}\rvert$) while the number of targets a solve needs falls as $\approx r/(2\lvert\mathcal{F}\rvert^2)$, so F4 has a preferred $\lvert\mathcal{F}\rvert$, and so does root finding. We therefore let each method pick its own. At 16 bits (Table 9), msolve's composed total is minimised at $\lvert\mathcal{F}\rvert = 16$ (1.41 s, composed); the root finder, measured in full at $\lvert\mathcal{F}\rvert = 120$, takes 6.9 ms (829 targets). The ratio at each method's own best is **205**×.

[1] Root time net of the table is ≈ 1.6 ms, at the resolution of the measurement; the ratio is correspondingly coarse.

Table 9: 16-bit, msolve per decide (paired-control net CPU) versus root finding per decide as $|\mathcal{F}|$ grows, and msolve's composed solve total. Artefact: same file, `section=best_fb_16bit`, `best_vs_best_16bit`.

$\lvert\mathcal{F}\rvert$	msolve / decide	root / decide	expected targets	msolve total (composed)
4	202.0 μs	0.7 μs	18 407	3.71 s
6	214.0 μs	0.8 μs	9 999	2.14 s
8	442.0 μs	0.8 μs	6 647	2.94 s
12	485.0 μs	0.8 μs	3 864	1.87 s
16	526.0 μs	0.7 μs	2 685	1.41 s
24	1.2 ms	1.1 μs	1 648	1.91 s
32	1.8 ms	1.0 μs	1 183	2.16 s
64	5.0 ms	1.4 μs	552	2.76 s
120	18.0 ms	6.4 μs	285	5.13 s

Why the end-to-end ratio is two orders of magnitude below the per-target ratio. The oracle is not the only cost in a solve. The root finder's relation collection is dominated by generating the targets themselves—a scalar multiplication per target—not by deciding them: at 16 bits the oracle is 16% of the root finder's total. Replacing a Gröbner oracle that costs milliseconds with one that costs a microsecond removes the oracle from the critical path, after which the ratio is set by what remains. The per-target numbers of §6.2–§6.4 measure how much faster the oracle got; this section measures how much faster the solve got; and the second is the smaller number, as it must be.

What was not measured. Buchberger beyond 16 bits and msolve beyond the fully-measured rows are composed, not timed end to end, and are marked as such. The $m = 3$ and $m = 4$ binary strategies and the NIST curves are not run end to end: at those sizes a complete solve is out of reach for every method (§7).

7 What the Measurements Do and Do Not Show

We state the boundary of the result in the same place as the result.

It is a constant-factor improvement to a subroutine. In the companion repository's classification of changes (advance, engineering, relabelling, accounting) this is *engineering*. The per-target ratios of §6.1–§6.4 are stage diagnostics; the end-to-end ratios of §6.5 are the speed-ups a user sees, and they are two orders of magnitude smaller because target generation and linear algebra remain. The number of decompositions index calculus must attempt is unchanged, and so is the probability that a random target decomposes. Over prime fields the cost remains on the order of $p/|\mathcal{F}|$ group operations per relation for $m = 2$, and the Gaudry–Diem analysis that prime-field index calculus does not beat Pollard rho for any m is untouched [2, 3]. The oracle was the slow step *per target*; it was never the reason prime-field index calculus fails to be competitive.

The exponential factor base on Koblitz curves is untouched. The Frobenius-invariant factor base relevant to $K_0/\mathbb{F}_{2^n}$ has dimension $\ell = \mathrm{ord}_2(n)$: $\ell = 52$ at $n = 53$, $\ell = 82$ at $n = 83$, so $|V| = 2^{52}$ or 2^{82}. No decomposition engine, Gröbner or root-finding, materializes such a factor base. The binary head-to-head of §6.1 lives where $\mathrm{ord}_2(n)$ is small; the $\mathbb{F}_{2^{53}}$ measurements of §6.3 deliberately bound the subspace to isolate the oracle cost from this wall. Raising n makes the factor base exponentially larger; it does not make decomposition harder, and faster decomposition does not help.

The NIST measurements are not a security result. On P-192 through P-521 only the per-target decomposition subroutine was measured, on a factor base of 6 or 64 points. The relation-collection and linear-algebra stages of index calculus remain exponentially out of reach at those sizes, and the asymptotics are unchanged. A fast decomposition oracle says nothing about the discrete-logarithm problem on these curves. We make no key-recovery, sub-rho, or deployed-curve claim, and we have written the benchmark output, the research note and this paper to prevent the result from being read as one.

The meet-in-the-middle cost grows with m. The direct method is $\Theta(|\mathcal{F}|^{m-1})$; the Gröbner method is designed for regimes where m is large enough that this is prohibitive. Our measurements are at $m = 2$ (and $m \leq 4$ for the binary table), where the comparison is decisively in favour of root finding. We do not claim the ordering persists at the large m that asymptotic index-calculus analyses contemplate, and at those m neither method is practical for the reasons above.

The per-target Gröbner baselines are the codebase's own; the end-to-end ones are not only that. In §6.1–§6.4 the binary baseline is the codebase's matrix-F4 on the identical Weil-descended system and the prime baseline is its Buchberger on the identical ideal; a state-of-the-art F4 is faster than that Buchberger by a large constant, which is exactly what §6.5 finds when it adds msolve: the prime ratio drops from $\sim 10^4\times$ (Buchberger) to $\sim 10^2\times$ (msolve). The msolve numbers are the ones to quote against F4. No implementation is faster than $\Theta(N)$ field operations on this ideal, since its generators already have degree N; and where a baseline capped its step budget the measured time is a lower bound, so the stated ratios understate rather than inflate its cost.

8 Related Work

Semaev [1] introduced summation polynomials and proposed the index-calculus application; Gaudry [2] and Diem [3] gave the first rigorous and heuristic analyses for curves over extension fields and established the negative result for prime fields. Faugère, Perret, Petit and Renault [6] and Petit and Quisquater [7] analysed the Weil-descended systems over $\mathbb{F}_{2^n}$ and the behaviour of F4/F5 on them; Galbraith and Gebregiyorgis [8] studied symmetries that reduce the system size. Joux and Vitse [9] developed decomposition over small-degree extension fields with a variant of F4 tuned to the summation-polynomial systems. The observation that summation-polynomial systems can be attacked by root finding rather than by Gröbner bases has appeared in the prime-field setting in the work of Amadori, Pintore and Sala [10], who propose a different polynomial model; our contribution is narrower and more concrete—for $m = 2$ the roots are given by the group law, so no polynomial model is needed at all—and is accompanied by an implementation, a fail-closed protocol, and measurements on standard curves. Montgomery's batched-inversion trick [14] is what makes the sweep cost one inversion. Our stance on claim hygiene follows the tradition of Kosters–Yeo [12] and others who have insisted that improvements to decomposition be reported together with their effect, or lack of effect, on the overall complexity.

9 Reproducibility

All code and raw measurements are in the companion repository (`aburan28/crypto`) under the following paths; the `crypto:` prefix used in table captions denotes a path in that repository at the commit this paper was built from.

Algorithms. `src/cryptanalysis/prime_fast.rs` (Algorithm 1, native field); `src/cryptanalysis/koblitz_index_calculus.rs` (`PairSumTable`, Algorithm 2; `DecompositionStrategy`).

Benchmarks.

`examples/semaev_root_vs_groebner_bench.rs` (§6.1, §6.2)
`examples/semaev_rootfind_scaling_bench.rs` (§6.3)
`examples/semaev_rootfind_nist_bench.rs` (§6.4)
`examples/semaev_rootfind_e2e_bench.rs` (§6.5; needs `msolve` on `PATH` for the external-F4 column)

Raw evidence (console transcripts and JSON).

`research/semaev_root_vs_groebner_20261012/`
`research/semaev_rootfind_scaling_20261012/`
`research/semaev_rootfind_nist_20261012/`
`research/semaev_rootfind_e2e_20261012/`
(`run1_*`: binary rows and Buchberger cells; `run2/`: msolve cells with paired-control accounting; see its `README.md`)

Research note with the same tables. `RESEARCH_SEMAEV_ROOTFIND_VS_GROEBNER.md`.

Lean formalization of Proposition 1 (Remark 1).
`papers/rootfinding-decomposition/lean/S3Roots.lean`
with `lakefile.toml`, `lean-toolchain` (Lean 4.34.1) and `lake-manifest.json` pinning Mathlib `v4.34.1`;
check with `lake exe cache get && lake build` there.

To reproduce:

```
cargo run --release --example semaev_root_vs_groebner_bench
cargo run --release --example semaev_rootfind_scaling_bench
F4_CONSTRUCTED=5 F4_RANDOM=5 \
  cargo run --release --example semaev_rootfind_nist_bench
cargo run --release --example semaev_rootfind_e2e_bench
E2E_SKIP_BINARY=1 E2E_FULL_BUDGET_S=300 \
  cargo run --release --example semaev_rootfind_e2e_bench
```

The NIST benchmark exposes its knobs as environment variables (`F4_FB`, `F4_CONSTRUCTED`, `F4_RANDOM`, `SCALE_FB`, `SCALE_RANDOM`) and prints them at the top of its output. The library test suite (`cargo test --release --lib`) passed with 2 247 tests at the commit the measurements were taken from.

10 Conclusion

For the decomposition sizes at which elliptic-curve index calculus is actually run in practice, the Gröbner-basis solve is not the right tool for the decomposition oracle: the variety it would compute is already parametrized by the group law. Replacing the solve with root finding—direct subtraction over prime fields, a pair-sum table over binary fields—is exact, complete, cheap to verify, and three to four orders of magnitude faster per target on every instance we measured, including the five NIST prime curves at their standard parameters. End to end, against a state-of-the-art F4 given its best configuration, complete prime-curve solves run 50–105× faster and the one binary instance large enough to amortize the pair table 122×; on the three smallest binary instances F4 is faster, and we say so. We have made it the default in our codebase and kept the Gröbner solvers as baselines. We have also tried to say clearly what this is not: it is a better constant in a subroutine, not a change to what index calculus can reach.

References

[1] I. Semaev. Summation polynomials and the discrete logarithm problem on elliptic curves. IACR Cryptology ePrint Archive, Report 2004/031, 2004.

[2] P. Gaudry. Index calculus for abelian varieties of small dimension and the elliptic curve discrete logarithm problem. *Journal of Symbolic Computation*, 44(12):1690–1702, 2009.

[3] C. Diem. On the discrete logarithm problem in elliptic curves. *Compositio Mathematica*, 147(1):75–104, 2011.

[4] J.-C. Faugère. A new efficient algorithm for computing Gröbner bases (F_4). *Journal of Pure and Applied Algebra*, 139(1–3):61–88, 1999.

[5] J.-C. Faugère. A new efficient algorithm for computing Gröbner bases without reduction to zero (F_5). In *ISSAC 2002*, pages 75–83. ACM, 2002.

[6] J.-C. Faugère, L. Perret, C. Petit, and G. Renault. Improving the complexity of index calculus algorithms in elliptic curves over binary fields. In *EUROCRYPT 2012*, LNCS 7237, pages 27–44. Springer, 2012.

[7] C. Petit and J.-J. Quisquater. On polynomial systems arising from a Weil descent. In *ASIACRYPT 2012*, LNCS 7658, pages 451–466. Springer, 2012.

[8] S. D. Galbraith and S. W. Gebregiyorgis. Summation polynomial algorithms for elliptic curves in characteristic two. In *INDOCRYPT 2014*, LNCS 8885, pages 409–427. Springer, 2014.

[9] A. Joux and V. Vitse. Elliptic curve discrete logarithm problem over small degree extension fields. *Journal of Cryptology*, 26(1):119–143, 2013.

[10] A. Amadori, F. Pintore, and M. Sala. On the discrete logarithm problem for prime-field elliptic curves. *Finite Fields and Their Applications*, 51:168–182, 2018.

[11] J. Berthomieu, C. Eder, and M. Safey El Din. msolve: A library for solving polynomial systems. In *ISSAC 2021*, pages 51–58. ACM, 2021.

[12] M. Kosters and S. L. Yeo. Notes on summation polynomials. arXiv:1503.08001, 2015.

[13] J. M. Pollard. Monte Carlo methods for index computation (mod p). *Mathematics of Computation*, 32(143):918–924, 1978.

[14] P. L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48(177):243–264, 1987.

[15] National Institute of Standards and Technology. Digital Signature Standard (DSS). FIPS PUB 186-4, July 2013.